BUSINESS REQUIREMENTS DOCUMENT

Grievance Hub

Digital Grievance Redressal System

Document Version	1.0
Status	Draft
Prepared By	Madhuri Kushwaha
Date	April 2026
Confidentiality	Internal / Restricted

1. Document Control

Version	Date	Author	Description
1.0	April 2026	Business Analyst	Initial draft — full BRD baseline

Approvals

Name	Role	Signature	Date
—	Project Sponsor		
—	Technical Lead		

2. Executive Summary

Grievance Hub is a full-stack web platform built to offer citizens a simple and transparent way to submit, monitor, and receive solutions for complaints related to public services. The application improves traditional complaint management by converting the entire process into a digital workflow—from complaint registration to final resolution—while using AI to automatically categorize issues and assign priority levels.

The system is designed for two main types of users: citizens, who raise complaints, and administrators, who review and resolve them. To ensure maximum accessibility and participation, the platform allows users to file complaints both anonymously and through authenticated accounts.

This document presents the detailed business requirements for Grievance Hub version 1.0, including its functional features, system design, data structures, API specifications, and acceptance criteria.

3. Business Objectives

The following objectives define the business value Grievance Hub is expected to deliver:

- centralized digital platform where citizens can easily lodge complaints related to public service departments.
- Minimize resolution time by implementing organized ticket assignment and enabling administrators to prioritize issues effectively.
- transparency and accountability by allowing citizens to monitor the real-time status of their complaints.
- Utilize AI to automatically categorize complaints and determine their priority, reducing the need for manual intervention.
- Support informed decision-making through an administrative dashboard that provides summarized grievance data and insights.
- Allow both anonymous and registered users to submit complaints, ensuring wider accessibility and participation.
- Build a scalable and maintainable system using modular design and standard architectural practices for future growth.

4. Project Scope

4.1 In Scope

- User registration, login, and JWT-based session management.
- Complaint submission with anonymous or authenticated modes; auto-generated unique Ticket IDs.
- Authenticated user dashboard showing personal complaint history and statuses.
- Admin dashboard with full grievance table, filtering, status update, and resolution remarks.
- RESTful API backend built with FastAPI; React SPA frontend with protected routing.
- MySQL database with structured relational schema for users, grievances, and tickets.
- AI-based ticket category classification: Complaint, Query, or Support.
- AI-based priority prediction: Low (Level 1), Medium (Level 2), High (Level 3).
- Citizen-facing ticket tracking page — search by Ticket ID without login.

4.2 Out of Scope

- Native mobile application (iOS / Android).
- Multi-language (i18n) or regional language support.
- Payment gateway or any financial transaction feature.
- Real-time chat or live support between citizen and admin.
- Email or SMS notification system for status updates.
- OAuth, SSO, or third-party social login integration.

5. Stakeholders & User Roles

Stakeholder	Role	Responsibility	Primary Interest
Citizen / End User	Complainant	Submit and track grievances	Easy submission, transparent status tracking
Admin / Officer	Resolver	Review, assign, and resolve tickets	Efficient workload management, resolution metrics
Developer / Intern	Builder	Design, build, and maintain the system	Clean architecture, portfolio demonstration
Project Sponsor	Approver	Define requirements, approve deliverables	On-time delivery, meets business objectives

6. Functional Requirements

6.1 Authentication & Authorization

ID	Requirement	Priority	Role
FR-01	System shall allow new users to register with a unique email and password.	Must Have	Citizen
FR-02	System shall authenticate users via email/password and return a signed JWT token.	Must Have	Citizen / Admin
FR-03	All protected endpoints shall reject requests with missing or expired JWT tokens with HTTP 401.	Must Have	All
FR-04	Admin role shall be seeded in the database; no self-registration path for admin exists.	Must Have	Admin

6.2 Grievance Submission

ID	Requirement	Priority	Role
FR-05	System shall allow citizens to submit a grievance with a title, description, and department tag.	Must Have	Citizen
FR-06	System shall support anonymous submission — no login required for basic complaint filing.	Must Have	Anonymous
FR-07	System shall auto-generate a unique alphanumeric Ticket ID upon successful submission.	Must Have	System
FR-08	Ticket ID shall be returned to the submitter immediately after submission for tracking purposes.	Must Have	System

6.3 AI Classification & Priority Prediction

ID	Requirement	Priority	Role
FR-09	On submission, AI module shall classify the ticket into one of: Complaint, Query, or Support.	Must Have	System (AI)
FR-10	AI module shall predict a priority level: Low (Level 1), Medium (Level 2), or High (Level 3).	Must Have	System (AI)
FR-11	Classification and priority shall be stored with the ticket record and visible to admins.	Must Have	System
FR-12	AI classification shall be handled via an external generative AI API (e.g., Gemini Flash, Groq); integration abstracted within ai_classifier.py for easy provider swap.	Must Have	System

6.4 Ticket Tracking

ID	Requirement	Priority	Role
FR-13	Any user (anonymous or authenticated) shall be able to look up ticket status by Ticket ID.	Must Have	All
FR-14	Authenticated citizens shall see all their submitted grievances on a personal dashboard.	Must Have	Citizen
FR-15	Ticket status lifecycle shall follow: Submitted → In Review → Resolved / Rejected.	Must Have	System

6.5 Admin Dashboard

ID	Requirement	Priority	Role
FR-16	Admin shall view all grievances in a paginated, sortable table with filters by status and priority.	Must Have	Admin
FR-17	Admin shall be able to update ticket status and add resolution remarks for any ticket.	Must Have	Admin
FR-18	Admin dashboard shall display aggregate stats: total tickets, resolved, pending, and high-priority count.	Should Have	Admin
FR-19	Admin shall be able to drill into a single grievance detail view showing full description, AI tags, and history.	Should Have	Admin

7. Non-Functional Requirements

ID	Category	Requirement	Target / Metric
NFR-01	Performance	API response time for standard CRUD operations	< 500ms at p95
NFR-02	Performance	AI classification response time per ticket	< 2 seconds
NFR-03	Security	Passwords shall be stored as bcrypt hash; never plaintext	bcrypt cost factor ≥ 10
NFR-04	Security	JWT tokens shall expire; all protected routes shall enforce RBAC	Token expiry ≤ 24h
NFR-05	Scalability	Backend shall follow modular router/service pattern for independent scaling	Architectural compliance
NFR-06	Usability	Frontend shall be responsive across desktop and tablet screen sizes	Works at 768px–1440px
NFR-07	Reliability	All API errors shall return structured JSON with appropriate HTTP codes	4xx/5xx with error body
NFR-08	Maintainability	Code shall follow separation of concerns: routers, services, schemas, models distinct	As per project structure

8. System Architecture

Grievance Hub follows a three-tier client-server architecture with clear separation between presentation, application, and data layers.

8.1 Presentation Layer (React + Vite SPA)

- Single-page application built with React and bundled via Vite.
- Global authentication state managed through AuthContext; protected routes enforced via ProtectedRoute component.
- Axios instance with request/response interceptors handles JWT injection and 401 token refresh logic.
- Pages: Home, Login, Register, Dashboard (citizen), SubmitGrievance, TrackTicket, AdminDashboard, GrievanceDetail.

8.2 Application Layer (FastAPI)

- RESTful API structured into four routers: auth_router, grievance_router, ticket_router, admin_router.
- Each router delegates business logic to a corresponding service layer, keeping endpoints thin and testable.
- JWT validation handled centrally via reusable dependency injection (dependencies.py).
- AI classification invoked via external API call (Gemini Flash / Groq) inside ai_classifier.py on every new ticket submission; response parsed and stored with the ticket.
- Environment configuration centralized in config.py; no hardcoded secrets in source code.

8.3 Data Layer (MySQL + SQLAlchemy)

- Relational MySQL database managed via SQLAlchemy ORM with session management in db.py.
- Three primary models: User, Grievance, Ticket — with foreign key relationships enforced at the ORM level.
- Database connection string and credentials loaded from environment variables via .env file.

9. API Endpoint Specification

Module	Method	Endpoint	Auth	Description
Auth	POST	/api/auth/register	None	Register a new citizen account
Auth	POST	/api/auth/login	None	Authenticate and receive JWT access token
Grievance	POST	/api/grievance/submit	Optional	Submit new grievance; triggers AI classifier
Grievance	GET	/api/grievance/my	Citizen JWT	Fetch all grievances for the authenticated user
Ticket	GET	/api/ticket/{ticket_id}	None	Track ticket status by Ticket ID (public)
Admin	GET	/api/admin/grievances	Admin JWT	List all grievances with optional filters
Admin	PATCH	/api/admin/grievance/{id}	Admin JWT	Update ticket status and add resolution remarks
Admin	GET	/api/admin/stats	Admin JWT	Retrieve aggregate dashboard statistics

10. Data Models

10.1 User

Field	Data Type	Constraint	Description
id	INT	PK, AI	Auto-increment primary key
email	VARCHAR(255)	UNIQUE, NN	User's unique email address
password_hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password string
role	ENUM	NOT NULL	Values: 'citizen' | 'admin'
created_at	DATETIME	DEFAULT NOW()	Timestamp of account creation

10.2 Grievance / Ticket

Field	Data Type	Constraint	Description
ticket_id	VARCHAR(20)	PK	Auto-generated unique alphanumeric Ticket ID
user_id	INT	FK, NULL	FK → User.id; NULL for anonymous submissions
title	VARCHAR(300)	NOT NULL	Short complaint title
description	TEXT	NOT NULL	Full complaint description (AI input source)
department	VARCHAR(100)	NOT NULL	Target department or service category
category	ENUM	NOT NULL	AI-classified: Complaint | Query | Support
priority	ENUM	NOT NULL	AI-predicted: Low | Medium | High
status	ENUM	NOT NULL	Submitted | In Review | Resolved | Rejected
admin_remarks	TEXT	NULL	Admin resolution notes; NULL until actioned
created_at	DATETIME	DEFAULT NOW()	Complaint submission timestamp
updated_at	DATETIME	ON UPDATE	Last modification timestamp (auto-updated)

11. Assumptions & Constraints

11.1 Assumptions

- Admin accounts are provisioned by seeding the database directly; no self-service admin registration flow exists in the application.
- The AI classification module will consume an external generative AI API (e.g., Google Gemini Flash or Groq) via a free-tier API key. No local model training is required.
- The system will be deployed on a local machine or cloud VM for demonstration and portfolio purposes; enterprise-grade high availability is out of scope for version 1.0.
- All users accessing the system have a modern browser (Chrome, Firefox, Edge) with JavaScript enabled.
- The development environment has Python 3.10+, Node.js 18+, and MySQL 8.0+ available.

11.2 Constraints

- Database must be MySQL; no substitution with PostgreSQL, MongoDB, or other storage engines is permitted in this version.
- Authentication is strictly email + password with JWT; no OAuth 2.0, Google Sign-In, or SSO is in scope.
- The AI utility (ai_classifier.py) shall use a free-tier generative AI API key (Gemini or Groq); the provider must offer a free quota sufficient for development and demo usage.
- Frontend must remain a client-side React SPA; server-side rendering frameworks (Next.js, Remix) are not in scope.
- The project must adhere to the defined folder structure as documented in the project repository.

12. Acceptance Criteria

ID	Feature	Acceptance Criteria	Status
AC-01	Registration	User can register with a new email; duplicate email returns HTTP 400.	Pending
AC-02	Login	Valid credentials return JWT token; invalid credentials return HTTP 401.	Pending
AC-03	Submission	Grievance submitted successfully; unique Ticket ID returned in response body.	Pending
AC-04	Anonymous	Complaint submittable without login; Ticket ID returned and trackable.	Pending
AC-05	AI Module	Every new ticket has non-null category and priority fields populated by AI classifier.	Pending
AC-06	Ticket Track	Public tracking page returns current ticket status and timestamps for valid Ticket ID.	Pending
AC-07	Admin Update	Admin can change ticket status and save remarks; citizen sees updated status on tracking page.	Pending
AC-08	RBAC	Citizen JWT cannot access /admin/* endpoints; returns HTTP 403.	Pending

13. Risks & Mitigations

ID	Risk	Likelihood	Mitigation
R-01	AI model accuracy too low for meaningful classification	Medium	Use a reliable free-tier AI API (Gemini Flash or Groq); implement fallback default values if API is unavailable; allow admin to manually override AI tags.
R-02	JWT token stolen or session hijacked	Low	Enforce short token expiry; store token in memory (not localStorage) on client side.
R-03	Database performance degrades under high ticket volume	Low	Add indexes on ticket_id, user_id, status, and created_at columns.
R-04	Anonymous complaints used to flood the system with spam	Medium	Implement rate limiting on the /grievance/submit endpoint via FastAPI middleware.
R-05	Scope creep extending delivery timeline	Medium	Strictly follow the in-scope definition in Section 4; defer new features to v1.1.

14. Glossary

Term	Definition
BRD	Business Requirements Document — formal specification of what a system must do.
JWT	JSON Web Token — a compact, URL-safe token used for stateless authentication.
RBAC	Role-Based Access Control — access governed by the role assigned to a user.
Ticket ID	A unique system-generated alphanumeric identifier assigned to each grievance submission.
SPA	Single Page Application — a web app that loads a single HTML page and dynamically updates content.
API	Application Programming Interface — a defined contract for how software components communicate.
ORM	Object-Relational Mapper — a library that maps database tables to application-level objects (e.g., SQLAlchemy).
Anonymous User	A citizen who submits a grievance without creating or logging into an account.